Sprint 4 — Final Report (Draft)

Title

AIMO Kaggle Project — Sprint 4

Abstract

Báo cáo này tóm t■t thi■t k■ h■ th■ng, c■ s■ lý thuy■t, tri■n khai Program-of-Thought (PoT) ■■ gi■i bài toán toán h■c d■ng AIME b■ng LLM k■t h■p sandboxed execution, phân tích l■i, và k■t qu■ ■ánh giá.

Table of Contents

---

1. Introduction

(Use `docs/report_outline.md` for structure and expand here.)

2. Related Work

(Summarize prior PoT / chain-of-thought literature briefly.)

3. LLM limitations for Mathematics

(From `docs/sprint4_theory.md`)

- S■ không nh■t quán s■ h■c.
- Thi■u kh■ n■ng th■c thi tính toán n■i t■i.
- Hallucination và l■i b■■c trung gian.

4. Program-of-Thought (PoT)

- Ý t■■ng: yêu c■u LLM sinh mã (Python) mô t■ b■■c tính.
- L■i ích: tính toán deterministic, d■ ki■m ch■ng, cho phép majority voting.
- R■i ro & mitigations: sandbox, timeout, parser robustness.

5. System Architecture and Rationale

- Mô t■ pipeline t■ng quát: Input → Prompt Template → LLM → Parser → Extracted Code / Raw Fallback → Sandboxed Executor → Normalizer → Evaluation & Voting → Report/Submission.

Pipeline diagram

See file: docs/pipeline_diagram.mmd

Rationale:

- PoT + sandbox execution ■■ t■ng ■■ chính xác và minh b■ch.
- H■ tr■ nhi■u engine (OpenAI / Google / vLLM) ■■ linh ho■t và t■n d■ng KV cache.
- K■t h■p raw numeric fallback và majority voting ■■ t■ng robustness.

6. Implementation Details

- `src/parser.py`: code extraction, `extract_numeric_answer` fallback.
- `src/executor.py`: `PythonExecutor` with stdout capture, timeout handling.
- `scripts/evaluate.py`: evaluation loop, records `prediction_method` and `is_correct`.
- `scripts/sprint4_analysis.py`: t■o bi■u ■■ và trích ví d■ case study.

7. KV Cache and vLLM considerations

(From theory file — KV cache gi■m chi phí cho nhi■u l■n inference v■i cùng context.)

8. Evaluation Methodology

Metrics used:

- Pass@1 (single-shot correctness)
- Pass@k / Majority voting (aggregate correctness across multiple predictions)
- Error taxonomy: Timeout, ExecutionError, ParseError, WrongResult, NoPrediction

Data:

- `data/evaluation_report.csv` — recorded fields: id, problem, true_answer, predicted_answer, prediction_method, is_correct, error_log

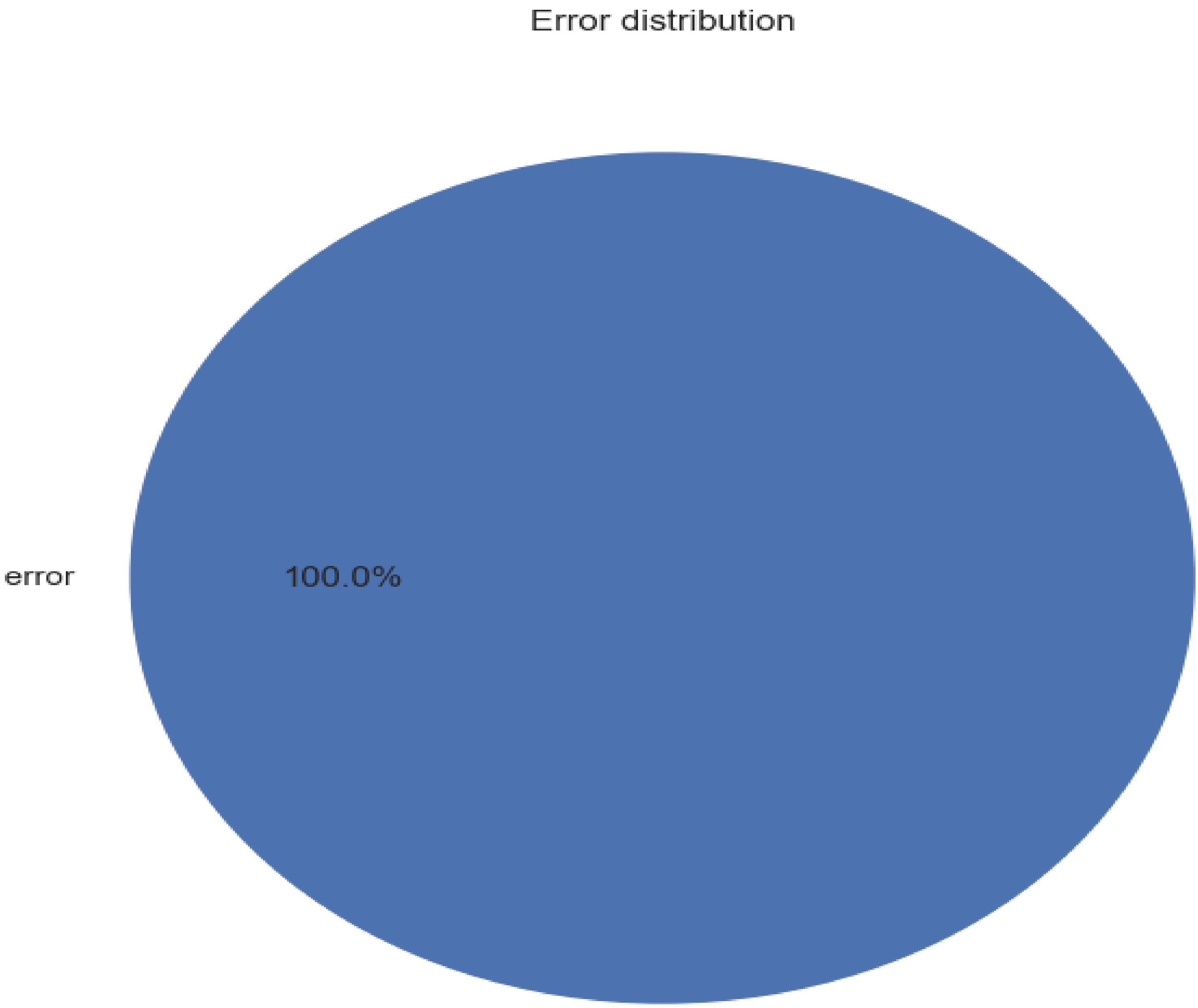
9. Results and Error Analysis

Summary statistics

(Include aggregated numbers — to be filled from analysis outputs.)

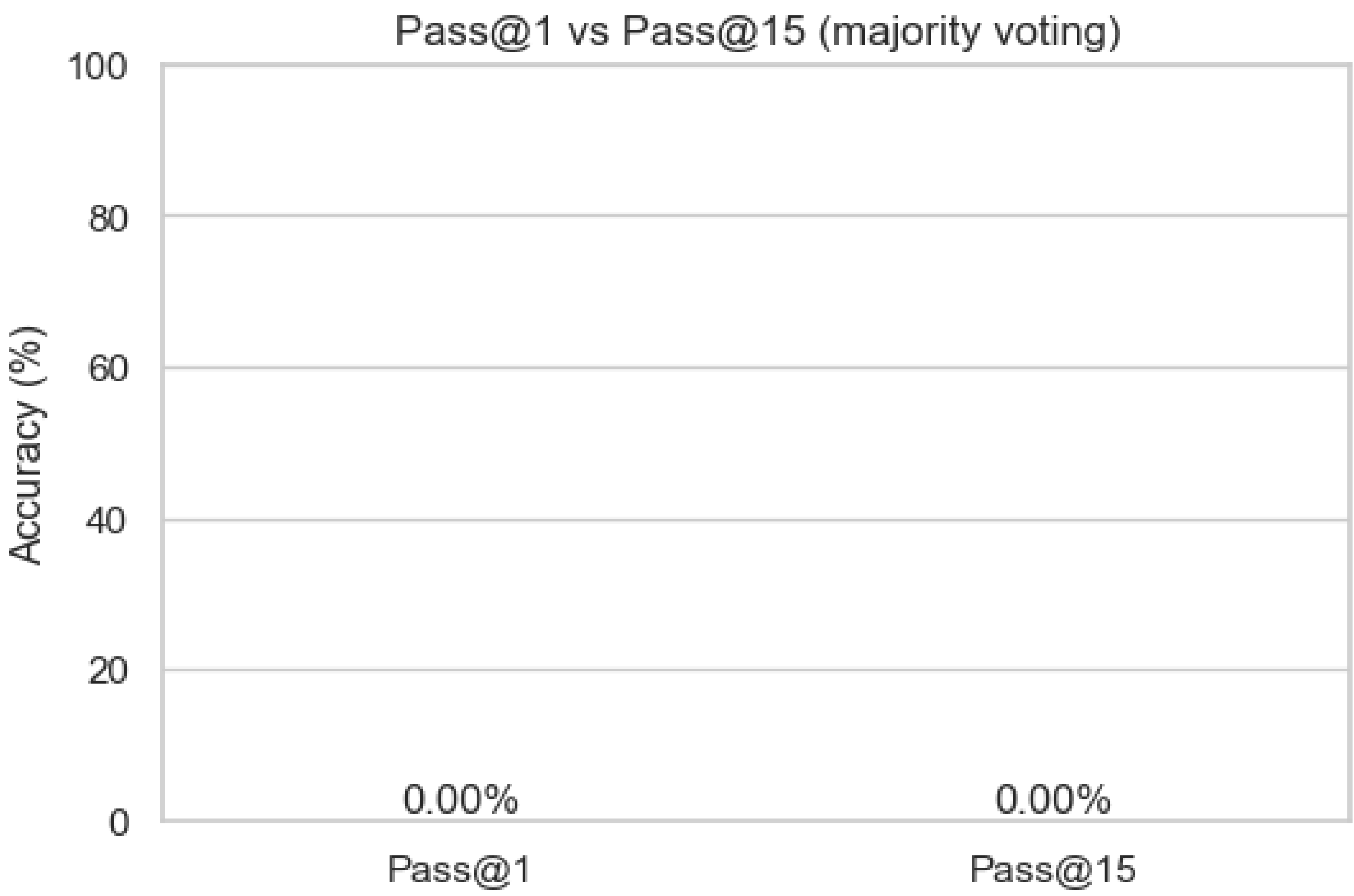
Error distribution

Include generated image: outputs/error_pie.png



Accuracy comparison

Include generated image: outputs/accuracy_comparison.png



10. Case Studies

(From `docs/case_studies.md`)

```
```

Include 2–3 highlighted examples where the model produced plausible code but an off-by-one or punctuation error led to wrong final answer.

11. Recommendations and Future Work

- Improve prompt templates to force explicit returned `result =` or `print()` statements.
- Expand parser to handle more LaTeX variants and fractional outputs.
- Use deterministic arithmetic engine for intermediate symbolic manipulation (SymPy) when needed.
- Explore micro-ensemble strategies and weighted voting.

12. Appendix

- Prompts (examples): see `scripts/` and `src/` prompt templates.
- How to reproduce analysis:

Run analysis script:

```bash
python scripts/sprint4_analysis.py --report data/evaluation_report.csv --out outputs
```

Convert markdown to PDF (suggestion):

```bash
pandoc docs/sprint4_report.md -o docs/sprint4_report.pdf --pdf-engine=xelatex --toc
```

End of draft.